

[Dashboard](#) / [Courses](#) / [Computer Science](#) / [2024](#) / [COMS3010A-OS3-S2-2024](#) / [Test 1 & Test 2](#) /

Started on	Monday, 14 October 2024, 2:31 PM
-------------------	----------------------------------

State	Finished
--------------	----------

Completed on	Monday, 14 October 2024, 3:21 PM
---------------------	----------------------------------

Time taken	50 mins 7 secs
-------------------	----------------

Marks	26.00/32.00
--------------	-------------

Grade	81.25 out of 100.00
--------------	----------------------------

Question 1

Correct

Mark 2.00 out of 2.00

A shared variable x , initialized to zero, is operated on by four concurrent processes W, X, Y, Z as follows. Process X reads x from [memory](#), increments by one, stores it to [memory](#) and then terminates. Each of the processes Y and Z reads x from [memory](#), decrements by two, stores it to [memory](#), and then terminates. Each process before reading acquires counting semaphore S and invokes the post operation on the semaphore S after storing x to [memory](#). What is the maximum possible value of x after all processes complete execution?

Process W	Process X	Process Y	Process Z
Wait (S)	Wait (S)	Wait (S)	Wait (S)
Read (x)	Read (x)	Read (x)	Read (x)
$x = x + 1;$	$x = x + 1;$	$x = x - 2;$	$x = x - 2;$
Write (x)	Write (x)	Write (x)	Write (x)
Signal (S)	Signal (S)	Signal (S)	Signal (S)

Answer: 2



Now, We have been asked the maximum possible value of x after all the processes complete execution. Clearly,

- Processes W and X increments the value of x .
- Processes Y and Z decrements the value of x .

To obtain the maximum value of x , the processes must execute in such a way that-

- Only the impact of the processes W and X remains on the value of x .
- The impact of processes Y and Z gets lost on the value of x .

Step-01:

- Process W arrives.
- It executes the wait(S) operation and the value of S decrements by 1. Now, $S = 1$.
- It reads the value $x = 0$.

- It gets preempted.

Step-02:

- Process Y arrives.
- It executes the wait(S) operation and the value of S decrements by 1. Now, $S = 0$.
- It reads the value $x = 0$.
- It decrements the value of x by 2. Now, $x = 0 - 2 = -2$.
- It writes the value $x = -2$ in the [memory](#).
- It executes the signal(S) operation and the value of S increments by 1. Now, $S = 1$.
- Now, execution of process Y is completed.

Step-03:

- Process Z arrives.
- It executes the wait(S) operation and the value of S decrements by 1. Now, $S = 0$.
- It reads the value $x = -2$.
- It decrements the value of x by 2. Now, $x = -2 - 2 = -4$.
- It writes the value $x = -4$ in the [memory](#).
- It executes the signal(S) operation and the value of S increments by 1. Now, $S = 1$.
- Now, execution of process Z is completed.

Step-04:

- Process W gets scheduled again.
- It resumes its execution from where it left.
- Before preemption it had already read the value $x = 0$.
- Now, it increments the value of x by 1. Now, $x = 0 + 1 = 1$.
- It writes the value $x = 1$ in the [memory](#).
- It executes the signal(S) operation and the value of S increments by 1. Now, $S = 2$.
- Now, execution of process W is completed.

Step-05:

- Process X arrives.
- It executes the wait(S) operation and the value of S decrements by 1. Now, $S = 1$.
- It reads the value $x = 1$.
- It increments the value of x by 1. Now, $x = 1 + 1 = 2$.

- It writes the value $x = 2$ in the [memory](#).
- It executes the signal(S) operation and the value of S increments by 1. Now, $S = 2$.
- Now, execution of process X is completed.

Thus,

- Final value of $x = 2$.
- This is the maximum possible value of x that can be achieved after executing all the 4 processes

The correct answer is: 2

Question 2

Correct

Mark 1.00 out of 1.00

Suppose that two threads have several critical sections, protected by different mutexes. The following shows their protection code.

```
code segment 1
lock(m1);
... /* code protected by m1 */
lock(m2);
... /* code protected by m2 */
unlock(m2);
unlock(m1)

code segment 2
lock(m2);
... /* code protected by m2 */
lock(m1);
... /* code protected by m1 */
unlock(m1);
unlock(m2)
```

Is that a sensible way to protect this critical code?

Select one:

- ☐ True
- ☒ False ✓

No, that is not a sensible way to control access to critical sections. Suppose that code segment 1 locks m2. Now each code segment will be forced to wait at a locked mutex. They deadlock.

The correct answer is 'False'.

Question 3

Correct

Mark 4.00 out of 4.00

Fill in the table below

- In the first 3 columns fill in the current pass value for the respective Processes A, B and C
- The last column must be completed to indicate at each row which processes would be scheduled
- If there is a tie select the job whose label appears earlier alphabetically

Stride Scheduling

Pass(A) - stride 100	Pass(B) - stride 50	Pass(C) - stride 150	Which Process Runs
0	0	0	A ✓
100 ✓	0 ✓	0 ✓	B ✓
100 ✓	50 ✓	0 ✓	C ✓
100 ✓	50 ✓	150 ✓	B ✓
100 ✓	100 ✓	150 ✓	A ✓
200 ✓	100 ✓	150 ✓	B ✓
200 ✓	150 ✓	150 ✓	B ✓
200 ✓	200 ✓	150 ✓	C ✓

Question **4**

Correct

Mark 1.00 out of 1.00

Which of the following [scheduling](#) algorithms can cause a problem known as "starvation"?

- ☐ a. None
- ☐ b. All
- ☒ c. Shortest Job Next (SJN) ✓
- ☐ d. First-Come, First-Served (FCFS)
- ☐ e. MLFQ with priority boost
- ☐ f. Round Robin (RR)

Your answer is correct.

The correct answer is:
Shortest Job Next (SJN)

Question 5

Correct

Mark 2.00 out of 2.00

Consider the following set of processes with their respective Length and arrival times. Using the First-Come, First-Served (FCFS) scheduling algorithm, what is the average waiting time for the processes?

Process Arrival Time Length

P1	0 ms	5 ms
P2	1 ms	3 ms
P3	2 ms	8 ms
P4	3 ms	6 ms

Please calculate your answer in ms, however, you must only type the numerical value, i.e. 4.3ms

Answer: 5.75



To calculate the **average waiting time** in the First-Come, First-Served (FCFS) scheduling algorithm:

1. Calculate completion time for each process:

- P1: 0 ms (arrival) + 5 ms = 5 ms
- P2: P1 completion (5 ms) + P2 (3 ms) = 8 ms
- P3: P2 completion (8 ms) + P3 (8 ms) = 16 ms
- P4: P3 completion (16 ms) + P4 (6 ms) = 22 ms

2. Calculate waiting time for each process:

- **Waiting time** = Completion time - Arrival time - Length
- P1: 5 ms - 0 ms - 5 ms = 0 ms
- P2: 8 ms - 1 ms - 3 ms = 4 ms
- P3: 16 ms - 2 ms - 8 ms = 6 ms
- P4: 22 ms - 3 ms - 6 ms = 13 ms

3. Average waiting time:

$$\text{Average waiting time} = \frac{(0+4+6+13)}{4} = 5.75 \text{ ms}$$

The correct answer is: 5.75

Question 6

Correct

Mark 2.00 out of 2.00

Consider the following three processes that arrive in a system at the specified times, along with the

- Process **P1** arrives at time **t=0** and has a **CPU burst of 10-time units**.
- **P2** arrives at **t=5** and has a **CPU burst of 2-time units**.
- **P3** arrives at **t=7** and has a **CPU burst of 3-time units**.

Assume that the processes execute only once for the duration of their CPU burst, and terminate i

Calculate the time of completion of the three processes under each of the following [scheduling](#) po
state the completion time of process P3. Assume there are no other processes in the scheduler's

For the preemptive policies, assume that a running process can be immediately preempted as so
the policy should decide to preempt).

(a) First Come First Serve ✓

(b) Shortest Remaining Time First (preemptive) ✓

(a) First Come First Serve =15

111111111122333 - p3 ends after 15-time units

(b) Shortest Remaining Time First (preemptive) =10

111112233311111 - p3 ends after 10-time units

Question 7

Correct

Mark 1.00 out of 1.00

A system uses a type of scheduler called a proportional-share scheduler. This scheduler makes su amount of CPU time, based on how many “tickets” it has. For example, if process A has 2 tickets, a twice as much CPU time as B. Note that a process cannot change how many tickets it has, and all I/O in this example).

Which of the following traces does not show the behaviour of a proportional-share scheduler?

- ☐ a. ABBBABBBABBBABBB
- ☐ b. AABAABAABAABAA
- ☐ c. ABABABABABAB
- ☒ d. None of the other options ✓
- ☐ e. AAAAAAAAAAABBBBB

Your answer is correct.

The correct answer is:

None of the other options

Question 8

Correct

Mark 2.00 out of 2.00

Given a job schedule-timeline of any length where 39 unique jobs appear at least once each. Using many misses would we experience in the worst case?

Answer: ✓

The correct answer is: 39.00

Question 9

Correct

Mark 1.00 out of 1.00

Consider N threads in a process. If one thread passes certain arguments to a function in the program, can that function be called by the other threads?

Select one:

☒ True ✓

☐ False

The correct answer is 'True'.

Question **10**

Correct

Mark 1.00 out of 1.00

Match the following terms with their corresponding statement

Takes all threads off the condition variables waiting list and marks them as eligible to run

Threads that operate on completely separate subsets of [memory](#)

Situations where two or more processes are reading or writing some shared data and the final result depends on the order of usage of the shared data, are called

Bugs that change behaviour or disappear when you try to examine them

Operations that run completely independently of any other processes

A process said to be in _____ state if it was waiting for an event that will never occur.

Your answer is correct.

The correct answer is: Takes all threads off the condition variables waiting list and marks them as eligible to run, Threads that operate on completely separate subsets of [memory](#) → independent, Situations where two or more processes are reading or writing some shared data and the final results depends on the order of usage of the shared data, are called race conditions, Bugs that change behaviour or disappear when you try to examine them → Heisenbugs, Operations that run completely independently of any other processes → Atomic, A process said to be in _____ state if it was waiting for an event that will never occur → deadlock

Question **11**

Correct

Mark 2.00 out of 2.00

Here is the classic semaphore version of the producer/consumer problem

```
void *producer(void *arg) { // core of producer
    for (i = 0; i < num; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        put(i);
        sem_post(&mutex);
        sem_post(&full);
    }
}

void *consumer(void *arg) { // core of consumer
    while (!done) {
        sem_wait(&full);
        sem_wait(&mutex);
        int tmp = get(i);
        sem_post(&mutex);
        sem_post(&empty);
        // do something with tmp ...
    }
}
```

For the following statements about this working solution, which statements are true, and which are

- (a) The semaphore full must be initialized to 0 ✓
- (b) The semaphore empty must be initialized to 0 ✓
- (c) The semaphore empty can be initialized to 0 ✓
- (d) The semaphore mutex must be initialized to 2 ✓

- (a) The semaphore full must be initialized to 0 - True The semaphore full must be initialized to 0 as there are; at the beginning, zero
- (b) The semaphore empty must be initialized to 0 - False Empty tracks how many empty buffers there should be initialized to the number of empty buffers. Thus, false.
- (c) The semaphore empty can be initialized to 0 - False There needs to be at least one buffer
- (d) The semaphore mutex must be initialized to 2 - False Mutex provides mutual exclusion and thus to work properly

Question **12**

Incorrect

Mark 0.00 out of 6.00

[Scheduling](#) policies can easily be depicted with some strings of characters. For example, let's say v then run job B for 8-time units.

Our string of this policy would look like this :

AAAAAAAAAABBBBBBBB

The priorities of the jobs can also be represented by a string of numbers. For example, for the above priority of 2 and Job B a priority of 1, which remained constant throughout the processing of the job this where each number corresponded to the priority of the job at that time step:

222222222211111111

Answer the following questions related to the scenario described below:

Scenario 1: Using multi-level feedback queue [scheduling](#) for Jobs A, B, C, D.

- Assume a 4-level MLFQ (having a priority descending from 3).
- Assume job A enters before B, they both don't require any I/O each with a runtime of 9 time-units
- Assume jobs C and D don't require any I/O each with a runtime of 5 time-units entering the system
- Assume a quantum length increases per level by 1 from 1 at the highest to 4 at the lowest
- Finally, Assume Round Robin is applied at each level
- Jobs are priority boosted to the highest priority if they spend 11-time units waiting at the lowest level scheduled job
- You MUST assume the arrival of new processes will interrupt the current process.

(a) What is the resultant [scheduling](#) string for Scenario 1? (Please don't include any spaces in your

ABAABBAAABCCCCCDDDDDABAABB ✖

(b) What is the resultant [scheduling](#) Priority string for Scenario 1? (Please don't include any spaces

3322221111322113221133222211 ✖

If the arrival of a new process interrupts the existing process.

	3AB		C		D		A
2	AABB		CC		DD		AA
1		AAAB		BB		CC	DD

3 A B C D A
0 BBB

If the arrival of a new process does not interrupt the existing process.

3 A B C D A B
2 A A B B C C D D A A B B
1 A A A B B B C C D D
0

Question 13

Correct

Mark 1.00 out of 1.00

In the Round Robin [scheduling](#) algorithm, what is the significance of the "time quantum"?

- ☐ a. It indicates the total response time of all processes.
- ☒ b. It is the maximum time a process can run before being preempted. ✓
- ☐ c. It represents the time taken by the shortest process.
- ☐ d. It indicates the total turnaround time of all processes.
- ☐ e. It determines the process with the highest priority.

Your answer is correct.

The correct answer is:

It is the maximum time a process can run before being preempted.

Question **14**

Correct

Mark 3.00 out of 3.00

Given a basic spin lock, assume locking the spin lock takes 5s (if no one is holding the lock); unlocking a context switch takes 10s and that a timeslice is 100s long

Assume this code sequence. executed by two threads on one CPU at roughly the same time:

- **mutex_lock();**
- **do_something();** - takes no time to execute
- **mutex_unlock();**
- **exit();** - takes no time to execute

For the following questions just give the numerical value in seconds and don't include the "s"

a) What is the best-case time for the two threads on one CPU to finish this code sequence

b) What is the worst-case time for the two threads to finish this code sequence? Assume that only 1 a maximum 

c) If the spin lock is instead changed to a queue-based lock how does that change the worst-case t

a) $4(5) + 10$ - here we have that both threads each perform a lock and unlock plus the one switch b

b) $4(5) + 3(10) + 100$ - here the worst case is that all 3 context switches occur, this would mean the waiting for the other to give up the lock, however, since we only have one CPU that wouldn't occur the entire time slice hence $+100$

c) Queues allow us to avoid that lengthy wait when one thread is waiting in vain for the other to rele

Question **15**

Correct

Mark 1.00 out of 1.00

Does the following code suffice to protect a critical section with two processes p_0 and p_1 running on a processor shared-memory machine. That is, does it meet *all* of the requirements for protecting a critical section of a shared variable.

Code for p_0

```
... /* non-critical-section code */

while(turn != 0) {}
    critical section
turn = 0;

... /* non-critical-section code */
```

Code for p_1

```
... /* non-critical-section code */

while(turn != 1) {}
    critical section
turn = 1;

... /* non-critical-section code */
```

Select one:

- ☐ True
- ☒ False ✓

No, it does not solve the critical section problem. When p_0 leaves its critical section, it sets $\text{turn} = 0$. When p_1 enters its critical section, it must wait because turn is not 1. p_1 will never be able to enter its critical section.

The correct answer is 'False'.

Question **16**

Correct

Mark 2.00 out of 2.00

Consider a system where multiple threads are trying to access a shared resource. The following details the system's behavior:

- The time to acquire a lock is 5 ms.
- The time to release a lock is 3 ms.
- Each thread spends 12 ms processing the shared resource while holding the lock.
- There are 4 threads, and only one thread can hold the lock at a time.
- You may assume there is no cost to switching between processes.
- If any Thread is in the process of acquiring a lock, that time does not count towards that thread's waiting time.

What is the total time that all threads spend waiting for the lock?

Please calculate your answer in ms, however, you must only type the numerical value, i.e. 4.3ms should be entered as 4.3.

Answer: 60



Each thread, except the first one, has to wait for the lock because only one thread can access the shared resource at a time. Calculate the waiting time for each thread:

- Thread 1:** Does not wait for the lock because it is the first to acquire it.
 - Waiting time: 0 ms
- Thread 2:** Waits for Thread 1 to acquire the lock, process the resource, and release the lock.
 - Time spent by Thread 1 = 5 ms (acquire) + 12 ms (processing) + 3 ms (release) = 20 ms
 - Waiting time for Thread 2 = 20 ms
- Thread 3:** Waits for both Thread 1 and Thread 2 to complete their work.
 - Time spent by Thread 2 = 20 ms
 - Waiting time for Thread 3 = 20 ms + 20 ms = 40 ms
- Thread 4:** Waits for Thread 1, Thread 2, and Thread 3 to complete their work.
 - Time spent by Thread 3 = 20 ms
 - Waiting time for Thread 4 = 40 ms + 20 ms = 60 ms

Total waiting time = Waiting time of Thread 2 + Waiting time of Thread 3 + Waiting time of Thread 4
= 20 ms + 40 ms + 60 ms = 120 ms

The correct answer is: 120

◀ Missed Work Form

Jump to...